

<Project Name> <Iteration/ Master> Test Plan

Version <1.0>

[Note: The following template is provided for use with the Rational Unified Process. Text enclosed in square brackets and displayed in blue italics (style=InfoBlue) is included to provide guidance to the author and should be deleted before publishing the document. A paragraph entered following this style will automatically be set to normal (style=Body Text).]

Revision History

Date	Version	Description	Author
<dd/mmm/yy>	<x.x>	<details>	<name>

Table of Contents

1.	Evaluation Mission and Test Motivation	3
2.	Target Test Items	3
3.	Test Approach	3
3.1	Testing Techniques and Types	3
3.1.1	Data and Database Integrity Testing	3
3.1.2	Function Testing	4
3.1.3	User Interface Testing	5
3.1.4	Performance Profiling	6
3.1.5	Load Testing	7
3.1.6	Security and Access Control Testing	8
3.1.7	Failover and Recovery Testing	9
3.1.8	Configuration Testing	11
4.	Deliverables	12
4.1	Test Evaluation Summaries	12
4.2	Reporting on Test Coverage	12
5.	Risks, Dependencies, Assumptions, and Constraints	12
6.	References	

<Iteration/ Master> Test Plan

1. Evaluation Mission and Test Motivation

[Provide an overview of the mission and motivation for the testing that will be conducted in this iteration.]

*[Provide a brief description of the background surrounding why the test effort defined by this **Test Plan** will be undertaken. Include information such as the key problem being solved, the major benefits of the solution, the planned architecture of the solution, and a brief history of the project..]*

[Provide a brief statement that defines the mission for the evaluation effort in the current iteration. This statement might incorporate one or more concerns including:

- find as many bugs as possible*
- find important problems, assess perceived quality risks*
- advise about perceived project risks*
- certify to a standard*
- verify a specification (requirements, design or claims)*
- advise about product quality, satisfy stakeholders*
- advise about testing*
- fulfill process mandates]*

2. Target Test Items

The listing below identifies those test items—software, hardware, and supporting product elements—that have been identified as targets for testing. This list represents what items will be tested.

[Provide a high level list of the major target test items. This list should include both items produced directly by the project development team, and items that those products rely on; for example, basic processor hardware, peripheral devices, operating systems, third-party products or components, and so forth. Consider grouping the list by category and assigning relative importance to each motivator.]

3. Test Approach

*[The Test Approach presents the recommended strategy for designing and implementing the required tests. Sections 2, Target Test Items, and Test Deliverables, identified **what** items will be tested and **what** types of tests would be performed. This section describes **how** the tests will be realized.*

One aspect to consider for the test approach is the techniques to be used. This should include an outline of how each technique can be implemented, both from a manual and/or an automated perspective, and the criterion for knowing that the technique is useful and successful. For each technique, provide a description of the technique and define why it is an important part of the test approach by briefly outlining how it helps achieve the Evaluation Mission or addresses the Test Motivators.

Another aspect to discuss in this section is the Fault or Failure models that are applicable and ways to approach evaluating them.

As you define each aspect of the approach, you should update Section Test Environment Configuration, to document the test environment configuration and other resources that will be needed to implement each aspect.]

1.1 Testing Techniques and Types

3.1.1 Data and Database Integrity Testing

[The databases and the database processes should be tested as an independent subsystem. This testing should test the subsystems without the target-of-test's User Interface as the interface to the data. Additional research into the DataBase Management System (DBMS) needs to be performed to identify the tools and techniques that may exist to support the testing identified in the following table.]

Technique Objective:	<i>[Exercise database access methods and processes independent of the UI so you can observe and log incorrect functioning target behavior or data corruption.]</i>
----------------------	--

Technique:	· <i>[Invoke each database access method and process, seeding each with valid and invalid data or requests for data.</i> · <i>Inspect the database to ensure the data has been populated as intended and all database events have occurred properly, or review the returned data to ensure that the correct data was retrieved for the correct reasons.]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>
Required Tools:	<i>[The technique requires the following tools:</i> • <i>Test Script Automation Tool</i> • <i>base configuration imager and restorer</i> • <i>backup and recovery tools</i> • <i>installation-monitoring tools (registry, hard disk, CPU, memory, and so forth)</i> • <i>database SQL utilities and tools</i> • <i>Data-generation tools]</i>
Success Criteria:	<i>[The technique supports the testing of all key database access methods and processes.]</i>
Special Considerations:	• <i>[Testing may require a DBMS development environment or drivers to enter or modify data directly in the databases.</i> • <i>Processes should be invoked manually.</i> • <i>Small or minimally sized databases (limited number of records) should be used to increase the visibility of any non-acceptable events.]</i>

1.1.1 Function Testing

[Function testing of the target-of-test should focus on any requirements for test that can be traced directly to use cases or business functions and business rules. The goals of these tests are to verify proper data acceptance, processing, and retrieval, and the appropriate implementation of the business rules. This type of testing is based upon black box techniques; that is verifying the application and its internal processes by interacting with the application via the Graphical User Interface (GUI) and analyzing the output or results. The following table identifies an outline of the testing recommended for each application.]

Technique Objective:	<i>[Exercise target-of-test functionality, including navigation, data entry, processing, and retrieval to observe and log target behavior.]</i>
Technique:	<i>[Execute each use-case scenario's individual use-case flows or functions and features, using valid and invalid data, to verify that:</i> · <i>the expected results occur when valid data is used</i> · <i>the appropriate error or warning messages are displayed when invalid data is used</i> · <i>each business rule is properly applied]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>

Required Tools:	<i>[The technique requires the following tools:</i> • <i>Test Script Automation Tool</i> • <i>base configuration imager and restorer</i> • <i>backup and recovery tools</i> • <i>installation-monitoring tools (registry, hard disk, CPU, memory, and so forth)</i> • <i>Data-generation tools]</i>
Success Criteria:	<i>[The technique supports the testing of:</i> · <i>all key use-case scenarios</i> · <i>all key features]</i>
Special Considerations:	<i>[Identify or describe those items or issues (internal or external) that impact the implementation and execution of function test.]</i>

1.1.1 User Interface Testing

[User Interface (UI) testing verifies a user's interaction with the software. The goal of UI testing is to ensure that the UI provides the user with the appropriate access and navigation through the functions of the target-of-test. In addition, UI testing ensures that the objects within the UI function as expected and conform to corporate or industry standards.]

Technique Objective:	<i>[Exercise the following to observe and log standards conformance and target behavior:</i> • <i>Navigation through the target-of-test reflecting business functions and requirements, including window-to-window, field-to-field, and use of access methods (tab keys, mouse movements, accelerator keys).</i> • <i>Window objects and characteristics can be exercised—such as menus, size, position, state, and focus.]</i>
Technique:	<i>[Create or modify tests for each window to verify proper navigation and object states for each application window and object.]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>
Required Tools:	<i>[The technique requires the Test Script Automation Tool.]</i>
Success Criteria:	<i>[The technique supports the testing of each major screen or window that will be used extensively by the end user.]</i>
Special Considerations:	<i>[Not all properties for custom and third-party objects can be accessed.]</i>

3.1.2 Performance Profiling

[Performance profiling is a performance test in which response times, transaction rates, and other time-sensitive requirements are measured and evaluated. The goal of Performance Profiling is to verify performance requirements have been achieved. Performance profiling is implemented and executed to profile and tune a target-of-test's performance behaviors as a function of conditions such as workload or hardware configurations.]

Note: *Transactions in the following table refer to “logical business transactions”. These transactions are defined as specific use cases that an actor of the system is expected to perform using the target-of-test, such as add or modify a given contract.]*

Technique Objective:	<i>[Exercise behaviors for designated functional transactions or business functions under the following conditions to observe and log target behavior and application performance data:</i> · <i>normal anticipated workload</i> · <i>anticipated worst-case workload]</i>
Technique:	· <i>[Use Test Procedures developed for Function or Business Cycle Testing.</i> · <i>Modify data files to increase the number of transactions or the scripts to increase the number of iterations that occur in each transaction.</i> · <i>Scripts should be run on one machine (best case to benchmark single user; single transaction) and should be repeated with multiple clients (virtual or actual, see Special Considerations below).]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>
Required Tools:	<i>[The technique requires the following tools:</i> • <i>Test Script Automation Tool</i> • <i>an application performance profiling tool, such as Rational Quantify</i> • <i>installation-monitoring tools (registry, hard disk, CPU, memory, and so on</i> • <i>resource-constraining tools; for example, Canned Heat]</i>
Success Criteria:	<i>The technique supports testing:</i> · <i>Single Transaction or single user: Successful emulation of the transaction scripts without any failures due to test implementation problems.]</i> · <i>Multiple transactions or multiple users: Successful emulation of the workload without any failures due to test implementation problems.]</i>
Special Considerations:	<i>[Comprehensive performance testing includes having a background workload on the server.</i> <i>There are several methods that can be used to perform this, including:</i> · <i>“Drive transactions” directly to the server, usually in the form of Structured Query Language (SQL) calls.</i> · <i>Create “virtual” user load to simulate many clients, usually several hundred. Remote Terminal Emulation tools are used to accomplish this load. This technique can also be used to load the network with “traffic”.</i> · <i>Use multiple physical clients, each running test scripts, to place a load on the system.</i> <i>Performance testing should be performed on a dedicated machine or at a dedicated time. This permits full control and accurate measurement.</i> <i>The databases used for Performance Testing should be either actual size or scaled equally.]</i>

3.1.3 Load Testing

[Load testing is a performance test that subjects the target-of-test to varying workloads to measure and evaluate the performance behaviors and abilities of the target-of-test to continue to function properly under these different workloads. The goal of load testing is to determine and ensure that the system functions

properly beyond the expected maximum workload. Additionally, load testing evaluates the performance characteristics, such as response times, transaction rates, and other time-sensitive issues).]

[**Note:** Transactions in the following table refer to “logical business transactions”. These transactions are defined as specific functions that an end user of the system is expected to perform using the application, such as add or modify a given contract.]

Technique Objective:	<i>[Exercise designated transactions or business cases under varying workload conditions to observe and log target behavior and system performance data.]</i>
Technique:	· <i>[Use Transaction Test Scripts developed for Function or Business Cycle Testing as a basis, but remember to remove unnecessary interactions and delays.</i> · <i>Modify data files to increase the number of transactions or the tests to increase the number of times each transaction occurs.</i> · <i>Workloads should include (for example, Daily, Weekly, Monthly and so forth) Peak loads.</i> · <i>Workloads should represent both Average as well as Peak loads.</i> · <i>Workloads should represent both Instantaneous and Sustained Peaks.</i> · <i>The Workloads should be executed under different Test Environment Configurations.]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>
Required Tools:	<i>[The technique requires the following tools:</i> • <i>Test Script Automation Tool</i> • <i>Transaction Load Scheduling and control tool</i> • <i>installation-monitoring tools (registry, hard disk, CPU, memory, and so on)</i> • <i>resource-constraining tools (for example, Canned Heat)</i> • <i>Data-generation tools]</i>
Success Criteria:	<i>[The technique supports the testing of Workload Emulation, which is the successful emulation of the workload without any failures due to test implementation problems.]</i>
Special Considerations:	· <i>[Load testing should be performed on a dedicated machine or at a dedicated time. This permits full control and accurate measurement.</i> · <i>The databases used for load testing should be either actual size or scaled equally.]</i>

3.1.4 Security and Access Control Testing

[Security and Access Control Testing focuses on two key areas of security:

- *Application-level security, including access to the Data or Business Functions*
- *System-level Security, including logging into or remotely accessing to the system.*

Based on the security you want, application-level security ensures that actors are restricted to specific functions or use cases, or they are limited in the data that is available to them. For example, everyone may be permitted to enter data and create new accounts, but only managers can delete them. If there is security at the data level, testing ensures that “user type one” can see all customer information, including financial data, however, “user two” only sees the demographic data for the same client.

System-level security ensures that only those users granted access to the system are capable of accessing

the applications and only through the appropriate gateways.]

Technique Objective:	<i>[Exercise the target-of-test under the following conditions to observe and log target behavior:</i> • <i>Application-level Security: an actor can access only those functions or data for which their user type is provided permissions.</i> • <i>System-level Security: only those actors with access to the system and applications are permitted to access them.</i>
Technique:	• <i>[Application-level Security: Identify and list each user type and the functions or data each type has permissions for.]</i> ○ <i>Create tests for each user type and verify each permission by creating transactions specific to each user type.</i> ○ <i>Modify user type and re-run tests for same users. In each case, verify those additional functions or data are correctly available or denied.</i> • <i>System-level Access: [See Special Considerations below]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>
Required Tools:	<i>[The technique requires the following tools:</i> • <i>Test Script Automation Tool</i> • <i>“Hacker” security breach and probing tools</i> • <i>OS Security Admin Tools]</i>
Success Criteria:	<i>[The technique supports the testing of for each known actor type the appropriate functions or data affected by security settings can be tested.]</i>
Special Considerations:	<i>[Access to the system must be reviewed or discussed with the appropriate network or systems administrator. This testing may not be required as it may be a function of network or systems administration.]</i>

3.1.5 Failover and Recovery Testing

[Failover and recovery testing ensures that the target-of-test can successfully failover and recover from a variety of hardware, software or network malfunctions with undue loss of data or data integrity.

For those systems that must be kept running failover testing ensures that, when a failover condition occurs, the alternate or backup systems properly “take over” for the failed system without any loss of data or transactions.

Recovery testing is an antagonistic test process in which the application or system is exposed to extreme conditions, or simulated conditions, to cause a failure, such as device Input/Output (I/O) failures, or invalid database pointers and keys. Recovery processes are invoked, and the application or system is monitored and inspected to verify proper application, or system, and data recovery has been achieved.]

Technique Objective:	<i>[Simulate the failure conditions and exercise the recovery processes (manual and automated) to restore the database, applications, and system to a desired, known, state. The following types of conditions are included in the testing to observe and log target behavior after recovery:</i> <i>· power interruption to the client</i> <i>· power interruption to the server</i> <i>· communication interruption via network servers</i> <i>· interruption, communication, or power loss to DASD (Dynamic Access Storage Devices) and DASD controllers</i> <i>· incomplete cycles (data filter processes interrupted, data synchronization processes interrupted)</i> <i>· invalid database pointers or keys</i> <i>· invalid or corrupted data elements in database]</i>
Technique:	<i>[The tests already created for Function and Business Cycle testing can be used as a basis for creating a series of transactions to support failover and recovery testing, primarily to define the tests to be run to test that recovery was successful.</i> <i>· Power interruption to the client: power the PC down.</i> <i>· Power interruption to the server: simulate or initiate power down procedures for the server.</i> <i>· Interruption via network servers: simulate or initiate communication loss with the network (physically disconnect communication wires or power down network servers or routers).</i> <i>· Interruption, communication, or power loss to DASD and DASD controllers: simulate or physically eliminate communication with one or more DASDs or controllers.</i> <i>Once the above conditions or simulated conditions are achieved, additional transactions should be executed and, upon reaching this second test point state, recovery procedures should be invoked.</i> <i>Testing for incomplete cycles uses the same technique as described above except that the database processes themselves should be aborted or prematurely terminated.</i> <i>Testing for the following conditions requires that a known database state be achieved.</i> <i>Several database fields, pointers, and keys should be corrupted manually and directly within the database (via database tools). Additional transactions should be executed using the tests from Application Function and Business Cycle Testing and full cycles executed.]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>
Required Tools:	<i>[The technique requires the following tools:</i> <i>• base configuration imager and restorer</i> <i>• installation monitoring tools (registry, hard disk, CPU, memory, and so on)</i> <i>• backup and recovery tools]</i>

Success Criteria:	<i>The technique supports the testing of:</i> • <i>One or more simulated disasters involving one or more combinations of the application, database, and system.</i> • <i>One or more simulated recoveries involving one or more combinations of the application, database, and system to a known desired state.]</i>
Special Considerations:	• <i>[Recovery testing is highly intrusive. Procedures to disconnect cabling (simulating power or communication loss) may not be desirable or feasible. Alternative methods, such as diagnostic software tools may be required.</i> • <i>Resources from the Systems (or Computer Operations), Database, and Networking groups are required.</i> • <i>These tests should be run after hours or on an isolated machine.]</i>

3.1.6 Configuration Testing

[Configuration testing verifies the operation of the target-of-test on different software and hardware configurations. In most production environments, the particular hardware specifications for the client workstations, network connections, and database servers vary. Client workstations may have different software loaded _for example, applications, drivers, and so on_ and, at any one time, many different combinations may be active using different resources.]

Technique Objective:	<i>[Exercise the target-of-test on the required hardware and software configurations to observe and log target behavior under different configurations and identify changes in configuration state.]</i>
Technique:	• <i>[Use Function Test scripts.</i> • <i>Open and close various non-target-of-test related software, such as Microsoft Excel and Word applications, either as part of the test or prior to the start of the test.</i> • <i>Execute selected transactions to simulate actors interacting with the target-of-test and the non-target-of-test software.</i> • <i>Repeat the above process, minimizing the available conventional memory on the client workstation.]</i>
Oracles:	<i>[Outline one or more strategies that can be used by the technique to accurately observe the outcomes of the test. The oracle combines elements of both the method by which the observation can be made and the characteristics of specific outcome that indicate probable success or failure. Ideally, oracles will be self-verifying, allowing automated tests to make an initial assessment of test pass or failure, however, be careful to mitigate the risks inherent in automated results determination.]</i>
Required Tools:	<i>[The technique requires the following tools:</i> • <i>base configuration imager and restore</i> • <i>installation monitoring tools (registry, hard disk, CPU, memory, and so on)]</i>
Success Criteria:	<i>[The technique supports the testing of one or more combinations of the target test items running in expected, supported deployment environments.]</i>
Special Considerations:	• <i>[What non-target-of-test software is needed, is available, and what is accessible on the desktop?</i> • <i>What applications are typically used?</i> • <i>What data are the applications running; for example, a large spreadsheet opened in Excel or a 100-page document in Word?</i> • <i>The entire system's netware, network servers, databases, and so on, also needs to be documented as part of this test.]</i>

4. Deliverables

[In this section, list the various artifacts that will be created by the test effort that are useful deliverables to the various stakeholders of the test effort. Don't list all work products; only list those that give direct, tangible benefit to a stakeholder and those by which you want the success of the test effort to be measured.]

4.1 Test Evaluation Summaries

[Provide a brief outline of both the form and content of the test evaluation summaries, and indicate how frequently they will be produced.]

1.1 Reporting on Test Coverage

[Provide a brief outline of both the form and content of the reports used to measure the extent of testing, and indicate how frequently they will be produced. Give an indication as to the method and tools used to record, measure, and report on the extent of testing.]

5. Risks, Dependencies, Assumptions, and Constraints

*[List any risks that may affect the successful execution of this **Test Plan**, and identify mitigation and contingency strategies for each risk. Also indicate a relative ranking for both the likelihood of occurrence and the impact if the risk is realized.]*

Risk	Mitigation Strategy	Contingency (Risk is realized)
Prerequisite entry criteria is not met.	<Tester> will define the prerequisites that must be met before Load Testing can start. <Customer> will endeavor to meet prerequisites indicated by <Tester>.	• Meet outstanding prerequisites • Consider Load Test Failure
Test data proves to be inadequate.	<Customer> will ensure a full set of suitable and protected test data is available. <Tester> will indicate what is required and will verify the suitability of test data.	• Redefine test data • Review Test Plan and modify components (that is, scripts) • Consider Load Test Failure
Database requires refresh.	<System Admin> will endeavor to ensure the Database is regularly refreshed as required by <Tester>.	• Restore data and restart • Clear Database

6. References

- Indicate the tool references (name, available at <<URL>>) used for testing
- Refer any data/ information in a standard format (eg. IEEE referencing style)
- For different algorithms/ techniques/ theories you can refer text books.
- For tools you can refer web pages.
- For similar work you can refer research paper articles that describe the work.
- You may include white paper articles for the description of technologies; web URL for the tool references. When you refer such a web page, you have to indicate the (Accessed on <<date>>)